

Infrastructure & Deployment Architecture Documentation

Personal Information

- **Name:** Hazem Hamdi Mahmoud Abdelqader
 - **University:** Nahda University (NUB)
 - **Faculty:** Faculty of Engineering
 - **Department:** Communications and Electronics Engineering
-

Contact Details

- **Phone:** [+20 106 238 6571](tel:+201062386571)
- **Email:** hazemhamdypersonal@gmail.com
- **GitHub:** github.com/Hazem-Hamd

Executive Summary

This document outlines the architectural decisions and implementation steps for a modern, self-hosted web infrastructure. The environment utilizes an Ubuntu Linux foundation, managed by Coolify (a self-hosted Platform-as-a-Service), and secured via Cloudflare's Zero Trust network (Cloudflare Tunnels). This architecture is specifically designed to host a Node.js and MySQL backend securely, without exposing local network ports to the public internet.

Phase 1: Server Provisioning & OS Installation

The "How"

The foundation of the infrastructure is a dedicated machine (or Virtual Machine) running **Ubuntu Server**.

1. The Ubuntu Server ISO is flashed to a bootable drive or mounted to a hypervisor.
2. During the installation wizard, static networking is configured (e.g., assigning a fixed local IP like 192.168.1.x) to ensure the server's internal address does not change upon reboot.
3. Only essential packages are installed initially to minimize the attack surface.

The "Why"

- **Stability and Support:** Ubuntu is the industry-standard Linux distribution for server environments. It offers predictable release cycles (LTS versions) and the most comprehensive community support and documentation.
 - **Resource Efficiency:** Running the "Server" edition (without a Desktop GUI) frees up critical CPU and RAM resources, dedicating them entirely to Docker containers, database operations, and Node.js execution.
-

Phase 2: Securing Remote Access

The "How"

Remote management is established via **SSH (Secure Shell)**.

1. The OpenSSH server package is installed during the OS setup.

2. Access is achieved from the administrative workstation using the standard terminal command: `ssh user@server_ip`.

The "Why"

- **Encrypted Communication:** SSH ensures that all commands, configurations, and data transferred between the administrative machine and the server are heavily encrypted. This prevents packet sniffing and man-in-the-middle attacks.
 - **Standardization:** It is the universal standard for Linux administration.
 - **Secure Port Forwarding:** By utilizing SSH tunnels, an administrator can interact with the MySQL database using local GUI tools (like DBeaver or Workbench) as if the database were running on their own machine, completely bypassing the need to open database ports on the server's firewall.
-

Phase 3: PaaS Installation & Database Setup

The "How"

Coolify is installed as the primary control plane for application deployment.

1. Coolify is installed via its official automated bash script

```
curl -fsSL https://cdn.coollabs.io/coolify/install.sh | sudo bash
```

2. Through the Coolify web dashboard, a **MySQL** database resource is deployed with a few clicks, generating necessary credentials and volumes.
3. The **Node.js** backend is subsequently deployed (either via Dockerfile, Docker Compose, or Git repository connection).

The "Why"

- **Developer Velocity:** Coolify abstracts the complex, manual management of Docker containers, reverse proxies (like Traefik/Nginx), and SSL certificates. It acts as a self-hosted Heroku, allowing for rapid deployment of Node.js services.
- **Internal Network Isolation:** Deploying both the Node.js API and the MySQL database within Coolify places them on the same internal Docker network. The Node.js app can communicate with MySQL directly via internal hostnames or IPs (e.g., localhost or the Docker gateway). The database is completely hidden from the outside world, creating a tight, secure backend architecture.

Phase 4: Domain Registration

The "How"

A custom domain (e.g., yourdomain.online) is acquired through **Namecheap**.

1. The desired domain name is searched and purchased via the Namecheap portal.

`https://www.namecheap.com/`

2. The domain is temporarily parked on Namecheap's default nameservers until the Cloudflare integration phase.

The "Why"

- **Brand Identity & Routing:** A custom domain is essential for creating clean, memorable API endpoints (e.g., api.yourdomain.online) rather than relying on raw IP addresses.
- **Registrar Reliability:** Namecheap provides cost-effective, straightforward domain management with reliable uptime for the initial registration phase.

Phase 5: DNS Configuration & Cloudflare Integration

The "How"

Download and install cloudflared:

```
wget https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-linux-amd64.deb
sudo dpkg -i cloudflared-linux-amd64.deb
```

Verify installation:

`cloudflared --version`

Cloudflare Tunnels.

1. **Changing Nameservers:** In the Namecheap dashboard, the default nameservers are replaced with the specific Cloudflare nameservers assigned to the account.

2. **Tunnel Creation:** In the Cloudflare Zero Trust dashboard, a new tunnel is created.
3. **Running the Daemon:** The cloudflared daemon is installed and authenticated on the Ubuntu server.
4. **Traffic Routing:** Public hostnames are configured within the Cloudflare dashboard to route specific traffic directly to local ports (e.g., routing app.yourdomain.online to http://localhost:3000 for the Node.js app, or routing TCP traffic for remote database access).

The "Why" (The Zero Trust Paradigm)

This is the most critical security feature of the architecture.

- **No Open Firewall Ports:** Traditional setups require opening ports (like 80 and 443) on the home or data center router (Port Forwarding). Cloudflare Tunnels reverse this paradigm. The cloudflared daemon creates an outbound, encrypted connection *from* the server *to* Cloudflare. As a result, the server's firewall remains completely closed to inbound traffic.
- **IP Obfuscation:** Because all traffic routes through Cloudflare's global edge network, the actual public IP address of the origin Ubuntu server is completely hidden. Attackers cannot port-scan or directly target the server.
- **Native SSL/TLS:** Cloudflare automatically provisions and renews SSL certificates for the domain, ensuring that all API payloads (such as authentication tokens) are encrypted in transit.
- **DDoS Mitigation:** Any malicious traffic spikes or Distributed Denial of Service attacks are absorbed by Cloudflare's massive edge network before they ever reach the internal Ubuntu server, ensuring high availability for the Node.js backend.